



Chapter 3. Python Packages

In this chapter, you'll learn how to package your Python projects for distribution. A *package* is a single file containing an archive of your code along with metadata that describes it, like the project name and version.

NOTE

Python folks use the word *package* for two distinct concepts. *Import packages* are modules that contain other modules. *Distribution packages* are archive files for distributing Python software—they're the subject of this chapter.

You can install a package into a Python environment using a package installer like `pip`. You can also upload it to a package repository for the benefit of others. The Python Software Foundation (PSF) operates a package repository known as the [Python Package Index \(PyPI\)](https://pypi.org/). If your package is on PyPI, anyone can install it by passing its project name to `pip install`.

Packaging your project makes it easy to share with others, but there's another benefit. When you install your package, it becomes a first-class citizen of a Python environment:

- The interpreter imports your modules from the environment—rather than an arbitrary directory on your filesystem, which may or may not work depending on how you invoke Python.
- Installers use the package metadata to ensure the environment matches the prerequisites of your package, such as the minimum Python version and any third-party packages it depends on.
- Installers can generate entry-point scripts that ensure your code always runs on the interpreter in the environment. Compare this to

hand-written Python scripts, which may run on the wrong Python version, or without the required third-party packages, or be unable to import their own modules.

In this chapter, I'll explain how you can package your Python projects and introduce you to tools that help with packaging tasks. The chapter has three parts:

- In the first part, I'll talk about the life of a Python package. I'll also introduce an example application that you'll use throughout this book. And I'll ask: why would you want to package your code at all?
- In the second part, I'll introduce Python's package configuration file, *pyproject.toml*, and tools for working with packages: `build`, `hatchling`, and `Twine`. The tools `pip`, `uv`, and `pipx` also make a reappearance. Finally, I'll introduce `Rye`, a project manager that ties these packaging tools together into a unified workflow. Along the way, you'll learn about build frontends and backends, wheels and sdist, editable installs, and the *src* layout.
- In the third part, I'll look at project metadata in detail—the various fields you can specify in *pyproject.toml* to define and describe your package, and how to make efficient use of them.

The Package Lifecycle

[Figure 3-1](#) shows the typical lifecycle of a package.

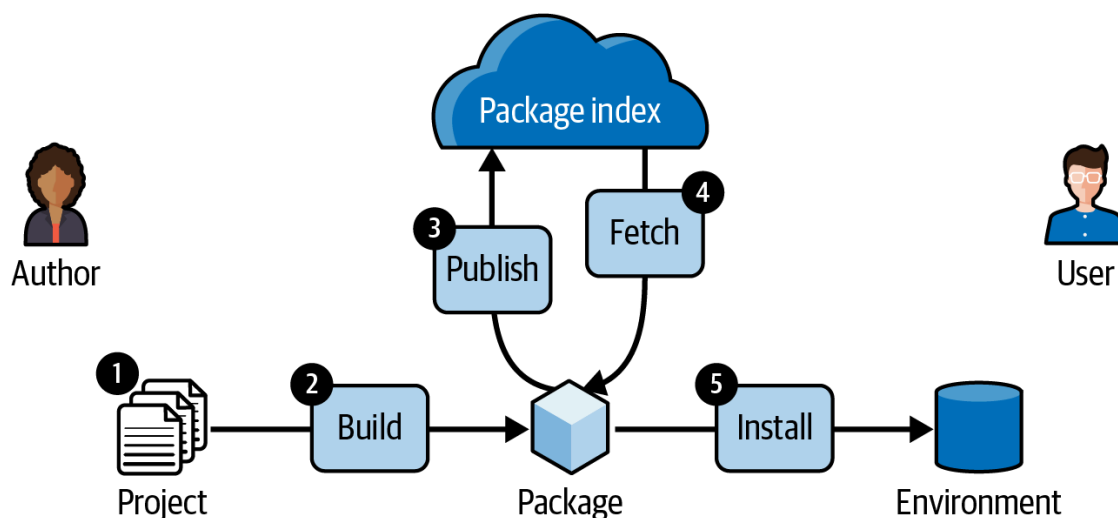


Figure 3-1. The package lifecycle: an author builds a project into a package and uploads it to a package index, and then a user downloads and installs the package into an environment.

- ❶ Everything starts with a *project*: the source code of an application, library, or other piece of software.
- ❷ As an author, your first step is to build a package from the project. The package is an installable artifact with a snapshot of your project at this point in time, identified by a unique name and version.
- ❸ Next, you publish the package to a well-known repository, such as PyPI.
- ❹ A user can now fetch your package by specifying its name and version.
- ❺ Finally, the user installs your package into their environment.

You can install a freshly built package directly into an environment without uploading it to a package repository first—for example, when you’re testing your package or when you’re its only user.

In real life, tools often combine fetching and installing, building and installing, and even building and publishing, into a single command.

An Example Application

Many applications start out as small, ad-hoc scripts. [Example 3-1](#) fetches a random article from Wikipedia and displays its title and summary in the console. The script restricts itself to the standard library, so it runs in any Python 3 environment.

Example 3-1. Displaying an extract from a random Wikipedia article

```
import json
import textwrap
import urllib.request
```

```
API_URL = "https://en.wikipedia.org/api/rest_v1/page/random/summary" ❶
```

```
def main():
    with urllib.request.urlopen(API_URL) as response: ❷
        data = json.load(response) ❸

    print(data["title"], end="\n\n") ❹
    print(textwrap.fill(data["extract"])) ❹

if __name__ == "__main__":
    main()
```

- ❶ The `API_URL` constant points to the REST API of the English Wikipedia—or more specifically, its `/page/random/summary` endpoint.
- ❷ The `urllib.request.urlopen` invocation sends an HTTP GET request to the Wikipedia API. The `with` statement ensures that the connection is closed at the end of the block.
- ❸ The response body contains the resource data in JSON format. Conveniently, the response is a file-like object, so the `json` module can load it like a file from disk.
- ❹ The `title` and `extract` keys hold the title of the Wikipedia page and a short plain text extract, respectively. The `textwrap.fill` function wraps the text so that every line is at most 70 characters long.

Store this script in a file called *random_wikipedia_article.py* and take it for a spin. Here's a sample run:

```
> py -m random_wikipedia_article
Jägersbleeker Teich
```

The Jägersbleeker Teich in the Harz Mountains of central Germany is a storage pond near the town of Clausthal-Zellerfeld in the county of Goslar in Lower Saxony. It is one of the Upper Harz Ponds that were created for the mining industry.

Why Packaging?

Sharing a script like [Example 3-1](#) doesn't require packaging. You can publish it on a blog or a hosted repository, or send it to friends by email or chat. Python's ubiquity, the “batteries included” approach of its standard library, and its nature as an interpreted language make this possible.

The ease of sharing modules with the world was a boon to Python's adoption in the early days. The Python programming language predates the advent of language-specific package repositories—PyPI didn't come about for more than a decade.¹

Distributing self-contained modules without packaging them seems like a great idea at first: you keep your projects free of packaging cruft. They require no separate artifacts, no intermediate steps like building, and no dedicated tooling. But using modules as the unit of distribution comes with limitations:

Projects composed of multiple modules

When your project outgrows a single-file script, you should break it up—but installing a collection of files is cumbersome for your users. Packaging lets you keep everything in a single file for distribution.

Projects with third-party dependencies

Python has a rich ecosystem of third-party packages, so you're standing on the shoulders of giants. But your users shouldn't need to worry about installing the correct version of every package you require. Packaging lets you declare dependencies on other packages, which installers satisfy automatically.

Finding the project

What's the repository URL of that useful module again? Or was it on a blog? If you publish a package on PyPI, your users only need to know its name to install the latest version. The situation is simi-

lar in a corporate environment, where developers' machines are configured to use a company-wide package repository.

Installing the project

Downloading and double-clicking a script won't work much of the time. You shouldn't need to place modules in arcane directories and perform a special dance so your script executes on the correct interpreter. Packaging lets users install your project with a single command, in a portable and safe way.

Updating the project

Users need to determine if the project is up-to-date and upgrade it to the latest version if it isn't. As an author, you need a way to let your users benefit from new features, bug fixes, and improvements. Package repositories let you publish a stream of releases for your project (a subset of the development snapshots you'd get from its code repository).

Running the project in the correct environment

Don't leave it to chance whether your program runs on a supported Python version with the required third-party packages. Package installers check and, where possible, satisfy your prerequisites. They also ensure that your code runs in the environment intended for it.

Binary extensions

Python modules written in a compiled language like C or Rust require a build step. Packaging lets you distribute prebuilt binaries for common platforms. Additionally, it lets you publish a source archive as a fallback; installers run the build step on the end user's machine.

Metadata

You can embed metadata inside a module, using attributes like `__author__`, `__version__`, or `__license__`. But then tools have to execute the module to read those attributes. Packages contain static metadata that any tool can read without running Python.

As you’ve seen, packaging solves many problems, but what’s the overhead? In short, you drop a declarative file named *pyproject.toml* into your project—a standard file that specifies the project metadata and its build system. In return, you get commands to build, publish, and install your package.

In summary, Python packages come with many advantages:

- You can easily install and upgrade them.
- You can publish them in a package repository.
- They can depend on other packages.
- They run in an environment that satisfies their requirements.
- They can contain multiple modules.
- They can contain prebuilt binary extensions.
- They can contain source distributions with automated build steps.
- They come with metadata that describes the package.

The pyproject.toml File

[Example 3-2](#) shows how to package the script from [Example 3-1](#) with a bare minimum of metadata—the project name and version—as well as an entry-point script. The project and the script use hyphens (`random-wikipedia-article`), while the module uses underscores (`random_wikipedia_article`). Place the module and the *pyproject.toml* file side by side in an empty directory.

Example 3-2. A minimal pyproject.toml file

```
[project]
name = "random-wikipedia-article"
version = "0.1"

[project.scripts]
random-wikipedia-article = "random_wikipedia_article:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

PyPI projects share a single namespace—their names aren’t scoped by the users or organizations owning the projects. Choose a unique name such as `random-wikipedia-article-{your-name}`, and rename the Python module accordingly.

At the top level, the *pyproject.toml* file can contain up to three sections—or *tables*, as the TOML standard calls them:

`[project]`

The `project` table holds the project metadata. The `name` and `version` fields are mandatory. For real projects, you should provide additional information, such as a description, the license, and the required Python version (see [“Project Metadata”](#)). The `scripts` section declares the name of the entry-point script and the function it should call.

`[build-system]`

The `build-system` table specifies how to build packages for the project (see [“Building Packages with build”](#))—specifically, which build tool your project uses. I’ve opted for `hatchling` here, which comes with [Hatch](#), a modern and standards-compliant Python project manager.

`[tool]`

The `tool` table stores configurations for each tool used by the project. For example, the Ruff linter reads its configuration from the `[tool.ruff]` table, while the type checker mypy uses `[tool.mypy]`.

THE TOML FORMAT

Python's project specification file uses *TOML* (Tom's Obvious Minimal Language), a cross-language format for configuration files that's both unambiguous and human-readable. The [TOML website](#) has a good introduction to the format.

Lists are termed *arrays* in TOML and use the same notation as Python:

```
requires = ["hatchling", "hatch-vcs"]
```

Dictionaries are known as *tables* and come in several equivalent forms. You can put the key/value pairs on separate lines, preceded by the table name in square brackets:

```
[project]
name = "foo"
version = "0.1"
```

Inline tables contain all key/value pairs on the same line:

```
project = { name = "foo", version = "0.1" }
```

You can also use dotted notation to create a table implicitly:

```
project.name = "foo"
project.version = "0.1"
```

You can load a TOML file using the standard `tomllib` module:

```
import tomllib

with open("pyproject.toml", mode="rb") as io: ❶
    data = tomllib.load(io)
```

- ❶ Always open the file in binary mode. TOML mandates UTF-8 character encoding; on older platforms, opening the file in text mode

can lead to garbled text (or *mojibake*, as it's affectionately known).

Python represents a TOML file as a dictionary, where keys are strings and values can be strings, integers, floats, dates, times, lists, or dictionaries. Here's what a *pyproject.toml* file looks like in Python:

```
{
  "project": {
    "name": "random-wikipedia-article",
    "version": "0.1",
    "scripts": {
      "random-wikipedia-article": "random_wikipedia_article:main"
    }
  },
  "build-system": {
    "requires": ["hatchling"],
    "build-backend": "hatchling.build"
  }
}
```

Building Packages with build

Let's create a package for your new project using `build`, a dedicated build frontend maintained by the Python Packaging Authority (PyPA). The PyPA is a group of volunteers that maintains a core set of software projects used in Python packaging.

NOTE

A *build frontend* is an application that orchestrates the build process for a Python package. Build frontends don't know how to assemble packaging artifacts from source trees. The tool that does the actual building is known as the *build backend*.

Open a terminal, change to the project directory, and invoke `build` with `pipx`:

```

$ pipx run build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - hatchling
* Getting build dependencies for sdist...
* Building sdist...
* Building wheel from sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - hatchling
* Getting build dependencies for wheel...
* Building wheel...
Successfully built random_wikipedia_article-0.1.tar.gz
and random_wikipedia_article-0.1-py2.py3-none-any.whl

```

By default, `build` creates two kinds of packages for a project, an sdist and a wheel (see [“Wheels and Sdists”](#)). You can find these files in the *dist* directory of your project.

As you can see in the previous output, `build` delegates the actual work to `hatchling`, the build backend you designated in [Example 3-2](#). A build frontend uses the `build-system` table to determine the build backend for the project ([Table 3-1](#)).

Table 3-1. The `build-system` table

Field	Type	Description
<code>requires</code>	Array of strings	The list of packages required to build the project
<code>build-backend</code>	String	The import name of the build backend in the format <code>package.module:object</code>
<code>build-path</code>	String	An entry for <code>sys.path</code> needed to import the build backend (optional)

[Figure 3-2](#) shows how the build frontend and the build backend collaborate to build a package.

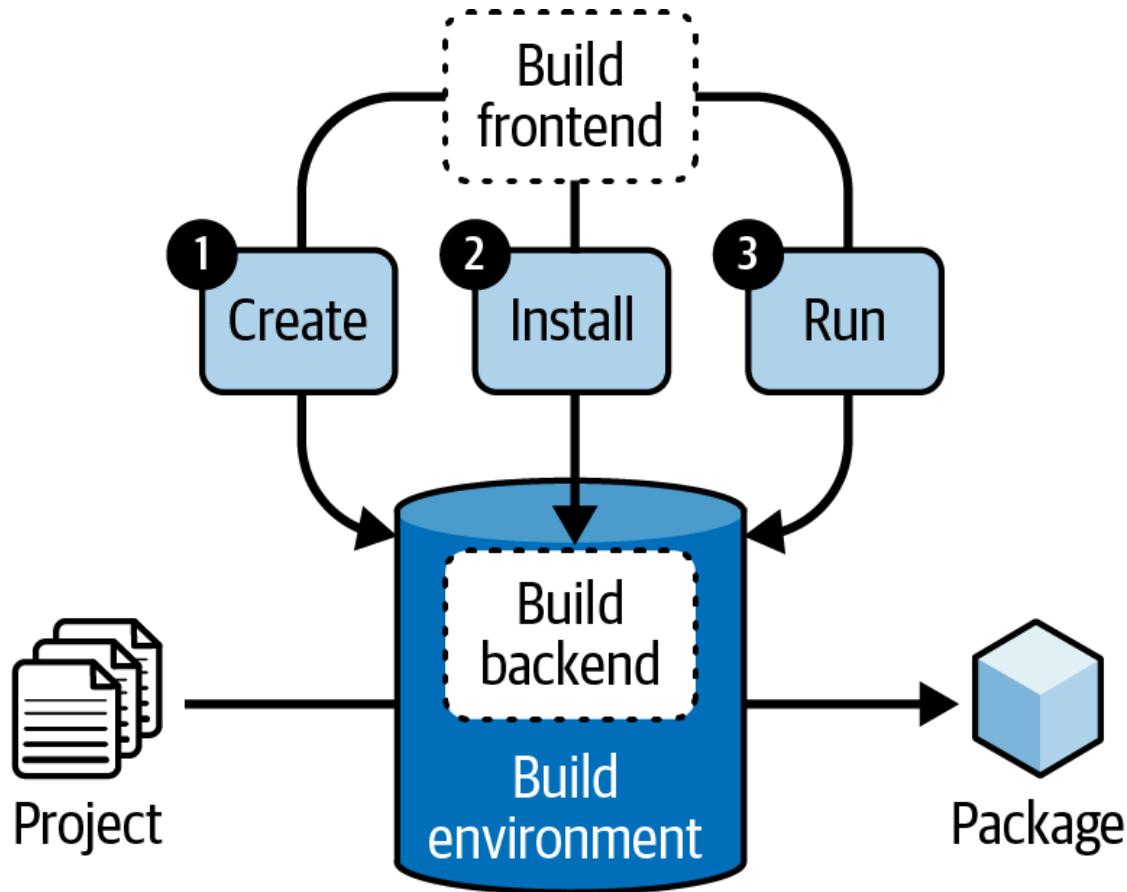


Figure 3-2. Build frontend and build backend

- ❶ The build frontend creates a virtual environment, the *build environment*.
- ❷ The build frontend installs the packages listed under `requires`: the build backend itself as well as, optionally, plugins for that backend. These packages are known as the *build dependencies*.
- ❸ The build frontend triggers the actual package build, in two steps. First, it imports the module or object declared in `build-backend`. Second, it invokes well-known functions for creating packages and related tasks, known as *build hooks*.

Here's a simplified version of the commands the build frontend performs when you build a wheel for your project:²

```
$ py -m venv buildenv
$ buildenv/bin/python -m pip install hatchling
$ buildenv/bin/python
>>> import hatchling.build as backend
>>> backend.get_requires_for_build_wheel()
```

```
[ ] # no additional build dependencies requested
>>> backend.build_wheel("dist")
'random_wikipedia_article-0.1-py2.py3-none-any.whl'
```

NOTE

Some build frontends let you build in your current environment. If you disable build isolation, the frontend only checks for build dependencies. If it installed them, the build and runtime dependencies of different packages might conflict.

Why separate the build frontend from the build backend? It means that tools can trigger package builds without knowing the intricacies of the build process. For example, package installers like pip and uv build packages on the fly when you install from a source directory (see [“Installing Projects from Source”](#)).

Standardizing the contract between build frontends and build backends has brought tremendous diversity and innovation to the packaging ecosystem. Build frontends include `build`, pip, and uv, the backend-agnostic Python project managers Rye, Hatch, and [PDM](#), and test automation tools like [tox](#). Build backends include those shipped with the project managers [Flit](#), Hatch, PDM, and [Poetry](#), the traditional build backend [setuptools](#), as well as exotic builders like [Maturin](#), a build backend for Python modules written in the Rust programming language, and [Sphinx Theme Builder](#), a build backend for Sphinx documentation themes ([Table 3-2](#)).

Table 3-2. Build backends

Project	requires ^{a}	build-backend
Flit	flit-core	flit_core.buildapi
Hatch	hatchling	hatchling.build
Maturin	maturin	maturin
PDM	pdm-backend	pdm.backend
Poetry	poetry-core	poetry.core.masonry.ap i
Setuptools	setuptools	setuptools.build_meta
Sphinx Theme Builder	sphinx-theme-builde r	sphinx_theme_builder

^{[a](#)} See the official documentation of each tool for any recommended version bounds.

Uploading Packages with Twine

It's time to publish your package. In this section, you'll use [TestPyPI](#), a separate instance of the Python Package Index intended for testing and experimentation. Just omit the `--repository` and `--index-url` options below to use the real PyPI.

First, register an account using the link on the front page of TestPyPI. Second, create an API token from your account page and copy the token to your preferred password manager. You can now upload the packages in *dist* using Twine, the official PyPI upload tool:

```
$ pipx run twine upload --repository=testpypi dist/*
Uploading distributions to https://test.pypi.org/legacy/
Enter your API token: *****
Uploading random_wikipedia_article-0.1-py2.py3-none-any.whl
Uploading random_wikipedia_article-0.1.tar.gz
```

View at:

<https://test.pypi.org/project/random-wikipedia-article/0.1/>

Congratulations, you have published your first Python package! Let's install the package from TestPyPI:

```
$ pipx install --index-url=https://test.pypi.org/simple random-wikipedia-article
installed package random-wikipedia-article 0.1, installed using Python 3.12.2
These apps are now globally available
- random-wikipedia-article
done!
```

You're now able to invoke your application from anywhere:

```
$ random-wikipedia-article
```

Installing Projects from Source

If you distribute packages for your project, it's a good idea to install these packages locally for development and testing. Running tests against an installed package rather than the source code means you're testing your project the way your users see it. And if you're working on a service, it helps keep development, staging, and production as similar as possible.

You *could* build a wheel with `build` and install it into a virtual environment:

```
$ pipx run build
$ uv venv
$ uv pip install dist/*.whl
```

There's a shortcut for this, though. Both `pip` and `uv` can install your project directly from a source directory, such as `.` for the current directory. Behind the scenes, they use the project's build backend to create a wheel for installation—they're build frontends just like `build`:

```
$ uv venv
$ uv pip install .
```

If your project comes with an entry-point script, you can also install it with `pipx`:

```
$ pipx install .
installed package random-wikipedia-article 0.1, installed using Python 3.12.2
These apps are now globally available
- random-wikipedia-article
```

During development, it saves time to see code changes reflected in the environment immediately, without repeatedly installing the project. You could import your modules directly from the source tree—but you’d lose all the benefits of packaging your project.

Editable installs achieve the best of both worlds by installing your package in a special way that redirects imports to the source tree (see [“Site Packages”](#)). You can think of this mechanism as a kind of “hot reloading” for Python packages. The `--editable` option (`-e`) works with `uv`, `pip`, and `pipx`:

```
$ uv pip install --editable .
$ py -m pip install --editable .
$ pipx install --editable .
```

Once you’ve installed your package in this way, you won’t need to reinstall it to see changes to the source code—only when you edit *pyproject.toml* to change the project metadata or add a third-party dependency.

Editable installs are modeled after the *development mode* feature from `setuptools`, if you’ve been around long enough to be familiar with it. But unlike `setup.py develop`, they rely on standard build hooks that any build backend can provide.

Project Layout

Dropping a *pyproject.toml* next to a single-file module is an appealingly simple approach. Unfortunately, this project layout comes with a serious footgun, as you'll see in this section. Let's start by breaking something in the project:

```
def main():  
    raise Exception("Boom!")
```

Before publishing your package, run a last smoke test with a locally built wheel:

```
$ pipx run build  
$ uv venv  
$ uv pip install dist/*.whl  
$ py -m random_wikipedia_article  
Exception: Boom!
```

A bug found is a bug fixed. After removing the offending line, verify that the program works as expected:

```
$ py -m random_wikipedia_article  
Cystiscus viaderi  
  
Cystiscus viaderi is a species of very small sea snail, a marine  
gastropod mollusk or micromollusk in the family Cystiscidae.
```

All good, time to cut a release! First, push your fix and a Git tag for the new version to your code repository. Next, use Twine to upload the wheel to PyPI:

```
$ pipx run twine upload dist/*
```

But, oh no—you never rebuilt the wheel. That bug is now in a public release! How could that happen?

Running your application with `py -m` protects you from inadvertently running an entry-point script from another installation (and it has the advantage of not requiring an active environment on macOS and Linux). But it also adds the current directory to the front of `sys.path` (see [“Site Packages”](#)). All along, you’ve been testing the module in your source tree, not the wheel you were going to publish!

You could set the `PYTHONSAFEPATH` environment variable and never think about this again—it’s an alias for `py -P` and omits the current directory from the module path. But that would leave your contributors out in the cold—and yourself, whenever you’re working on another machine.

Instead, move your module out of the top-level directory so folks can’t import it by mistake. By convention, Python source trees go into the *src* directory—which is why the arrangement is known as *src layout* in the Python community.

At this point, it also makes sense to convert your single-file module into an import package. Replace the *random_wikipedia_article.py* file by a *random_wikipedia_article* directory with a `__init__.py` module.

Placing your code in an import package is mostly equivalent to having it in a single-file module—but there’s one difference: you can’t run the application with `py -m random_wikipedia_article` unless you also add the special `__main__.py` module to the package ([Example 3-3](#)).

Example 3-3. The `__main__.py` module

```
from random_wikipedia_article import main

main()
```

The `__main__.py` module replaces the `if __name__ == "__main__":` block in `__init__.py`. Remove that block from the module.

This leaves you with a classic initial project structure:

```
random-wikipedia-article
├── pyproject.toml
└── src
    └── random_wikipedia_article
        ├── __init__.py
        └── __main__.py
```

An import package makes it easier for your project to grow: you can move code into separate modules and import it from there. For example, you could extract the code that talks to the Wikipedia API into a function `fetch`. Next, you might move the function to a module `fetch.py` in the package. Here's how you'd import the function from `__init__.py`:

```
from random_wikipedia_article.fetch import fetch
```

Eventually, `__init__.py` will contain only `import` statements for your public API.

Managing Packages with Rye

Many modern programming languages come with a single tool for building, packaging, and other development tasks. You may wonder how the Python community ended up with so many packaging tools with disjoint responsibilities.

The answer has to do with the nature and history of the Python project: Python is a decentralized open source project driven by a community of thousands of volunteers, with a history spanning more than three decades of organic growth. This makes it hard for a single packaging tool to cater to all demands and become firmly established.³

Python's strength lies in its rich ecosystem—and interoperability standards promote this diversity. As a Python developer, you have a choice of small single-purpose tools that play well together. This approach ties in with the Unix philosophy of “Do one thing, and do it well.”

But the Unix approach is no longer your only choice. *Python project managers* provide a more integrated workflow. Among the first, Poetry (see [Chapter 5](#)) has set itself the goal of reinventing Python packaging and pioneered ideas such as static metadata and cross-platform lock files.

Rye, a Python project manager written in Rust, chooses a different path. It provides a unified development experience on top of the widely used single-purpose tools you’ve already seen (and are about to see) in this book. Started by Armin Ronacher as a private project and first released to the public in 2023, it’s now under the stewardship of Astral, the company behind Ruff and uv. Please see Rye’s [official documentation](#) for installation instructions.

Your first step with Rye is initializing a new project with `rye init`. If you don’t pass the project name, Rye uses the name of the current directory. Use the `--script` option to include an entry-point script:

```
$ rye init random-wikipedia-article --script
```

Rye initializes a Git repository in *random-wikipedia-article* and populates it with a *pyproject.toml* file, an import package `random_wikipedia_article` in the *src* directory, a *README.md* with the project description, a *.python-version* file with the default Python version, and a virtual environment in *.venv*. Rye supports various build backends, with `hatchling` as the default choice:

```
random-wikipedia-article
├── .git
├── .gitignore
├── .python-version
├── .venv
├── README.md
├── pyproject.toml
├── src
│   └── random_wikipedia_article
│       ├── __init__.py
│       └── __main__.py
```

Many of Rye’s commands are frontends to tools that have become a de facto standard in the Python world—or that promise to become one in the future. The command `rye build` creates packages with `build`, the command `rye publish` uploads them using Twine, and the command `rye sync` performs an editable install using `uv`:

```
$ rye build
$ rye publish -r testpypi --repository-url https://test.pypi.org/legacy/
$ rye sync
```

There’s much more to `rye sync`, though. Rye manages private Python installations using the Python Standalone Builds project (see [“A Brave New World: Installing with Hatch and Rye”](#)), and `rye sync` fetches each Python version on first use. The command also generates a lock file for the project dependencies and synchronizes the environment with that file (see [Chapter 4](#)).

Wheels and Sdists

In [“Building Packages with build”](#), `build` created two packages for your project:

- `random_wikipedia_article-0.1.tar.gz`
- `random_wikipedia_article-0.1-py2.py3-none-any.whl`

These artifacts are known as *wheels* and *sdists*. Wheels are ZIP archives with a `.whl` extension, and they’re *built distributions*—for the most part, installers extract them into the environment as-is. Sdists, by contrast, are *source distributions*: they’re compressed archives of the source code with packaging metadata. Sdists require an additional build step to produce an installable wheel.

The name “wheel” for a Python package is a reference to wheels of cheese. PyPI was originally known as the *Cheese Shop*, after the Monty Python sketch about a cheese shop with no cheese whatsoever. (These days, PyPI serves over a petabyte of packages per day.)

The distinction between source distributions and built distributions may seem strange for an interpreted language like Python. But you can also write Python modules in a compiled language. In this situation, source distributions provide a useful fallback when no prebuilt wheels are available for a platform.

As a package author, you should build and publish both sdist and wheels for your releases. This gives users a choice: they can fetch and install the wheel if their environment is compatible (which is always the case for a pure Python package)—or they can fetch the sdist and build a wheel from it locally (see [Figure 3-3](#)).

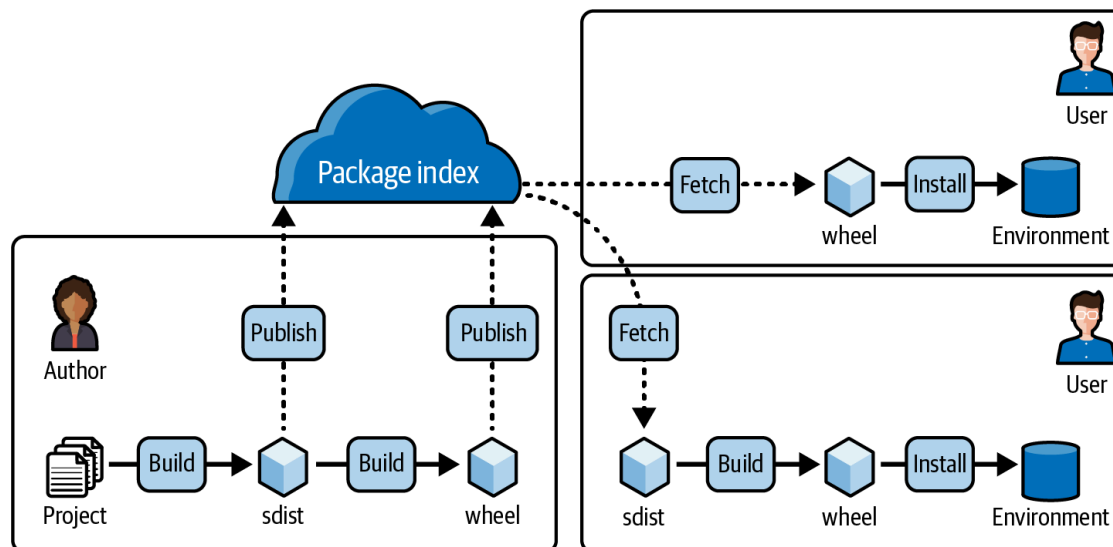


Figure 3-3. Wheels and sdists

For consumers of packages, sdists come with a few caveats. First, the build step involves arbitrary code execution, which can be a security concern.⁴ Second, installing wheels is much faster than installing sdists, especially for legacy *setup.py*-based packages. Lastly, users may encounter confusing build errors for extension modules if they don’t have the required build toolchain on their system.

Generally, a pure Python package has a single sdist and a single wheel for a given release. Binary extension modules, on the other hand, commonly come in wheels for a range of platforms and environments.

TIP

If you're an author of extension modules, check out the `cibuildwheel` project: it automates the building and testing of wheels across multiple platforms, with support for GitHub Actions and various other continuous integration (CI) systems.

Installers select the appropriate wheel for an environment using three so-called *compatibility tags* that are embedded in the name of each wheel file:

Python tag

The target Python implementation

ABI tag

The target *application binary interface* (ABI) of Python, which defines the set of symbols that binary extension modules can use to interact with the interpreter

Platform tag

The target platform, including the processor architecture

Pure Python wheels are usually compatible with any Python implementation, do not require a particular ABI, and are portable across platforms. Wheels express such wide compatibility using the tags `py3-none-any`.

Wheels with binary extension modules, on the other hand, have more stringent compatibility requirements. Take a look at the compatibility tags of these wheels, for example:

`numpy-1.24.0-cp311-cp311-macosx_10_9_x86_64.whl`

NumPy is a fundamental library for scientific computing. Its wheel targets a specific Python implementation and version (CPython 3.11), operating system release (macOS 10.9 and above), and processor architecture (x86-64).

`cryptography-38.0.4-cp36-abi3-manylinux_2_28_x86_64.whl`

Cryptography is another fundamental library, with an interface to cryptographic algorithms. Its wheel demonstrates two ways to reduce the build matrix for binary distributions: the *stable ABI* is a restricted set of symbols that are guaranteed to persist across Python feature versions (`abi3`), and the `manylinux` tag advertises

compatibility with a particular C standard library implementation (glibc 2.28 and above) across a wide range of Linux distributions.

If you're curious, you can extract a wheel using the `unzip` utility to see the files that installers place in the *site-packages* directory. Execute the following commands in a shell on Linux or macOS—preferably inside an empty directory. If you're on Windows, you can follow along using the Windows Subsystem for Linux (WSL):

```
$ py -m pip download attrs
$ unzip *.whl
$ ls -l
attr
attrs
attrs-23.2.0.dist-info
attrs-23.2.0-py3-none-any.whl

$ head -n4 attrs-23.2.0.dist-info/METADATA
Metadata-Version: 2.1
Name: attrs
Version: 23.2.0
Summary: Classes Without Boilerplate
```

Besides the import packages (named `attr` and `attrs` in this case), the wheel contains a *.dist-info* directory with administrative files. The *METADATA* file in this directory contains the *core metadata* for the package, a standardized set of attributes that describe the package for the benefit of installers and other packaging tools. You can access the core metadata of installed packages at runtime using the standard library:

```
$ uv pip install attrs
$ py
>>> from importlib.metadata import metadata
>>> metadata("attrs")["Version"]
23.2.0
>>> metadata("attrs")["Summary"]
Classes Without Boilerplate
```

In the next section, you'll see how to embed core metadata in your own packages, using the `project` table in *pyproject.toml*.

NOTE

The core metadata standards predate *pyproject.toml* by many years. Most project metadata fields correspond to a core metadata field, but their names and syntax differ slightly. As a package author, you can safely ignore this translation and focus on the project metadata.

Project Metadata

Build backends write out core metadata fields based on what you specify in the `project` table of *pyproject.toml*. [Table 3-3](#) provides an overview of all the fields you can use in the `project` table.

Table 3-3. The `project` table

Field	Type	Description
<code>name</code>	String	The project name
<code>version</code>	String	The version of the project
<code>description</code>	String	A short description of the project
<code>keywords</code>	Array of strings	A list of keywords for the project
<code>readme</code>	String or table	A file with a long description of the project
<code>license</code>	Table	The license governing the use of this project
<code>authors</code>	Array of tables	The list of authors
<code>maintainers</code>	Array of tables	The list of maintainers
<code>classifiers</code>	Array of strings	A list of classifiers describing the project
<code>urls</code>	Table of strings	The project URLs
<code>dependencies</code>	Array of strings	The list of required third-party packages
<code>optional-dependencies</code>	Table of arrays of strings	Named lists of optional third-party packages (<i>extras</i>)
<code>scripts</code>	Table of strings	Entry-point scripts
<code>gui-scripts</code>	Table of strings	Entry-point scripts providing a graphical user interface
<code>entry-points</code>	Table of tables of strings	Entry-point groups
<code>requires-python</code>	String	The Python version required by this project
<code>dynamic</code>	Array of strings	A list of dynamic fields

Two fields are essential and mandatory for every package: `project.name` and `project.version`. The project name uniquely identifies the project itself. The project version identifies a *release*—a published snapshot of the project during its lifetime. Besides the name and version, there are a number of optional fields you can provide, such as the author and license, a short text describing the project, or third-party packages used by the project (see [Example 3-4](#)).

Example 3-4. A `pyproject.toml` file with project metadata

```
[project]
name = "random-wikipedia-article"
version = "0.1"
description = "Display extracts from random Wikipedia articles"
keywords = ["wikipedia"]
readme = "README.md" # only if your project has a README.md file
license = { text = "MIT" }
authors = [{ name = "Your Name", email = "you@example.com" }]
classifiers = ["Topic :: Games/Entertainment :: Fortune Cookies"]
urls = { Homepage = "https://yourname.dev/projects/random-wikipedia-article" }
requires-python = ">=3.8"
dependencies = ["httpx>=0.27.0", "rich>=13.7.1"]
```

In the following sections, I'll take a closer look at the various project metadata fields.

Naming Projects

The `project.name` field contains the official name of your project:

```
[project]
name = "random-wikipedia-article"
```

Your users specify this name to install the project with `pip`. This field also determines your project's URL on PyPI. You can use any ASCII letter or digit to name your project, interspersed with periods, underscores, and hyphens. Packaging tools normalize project names for comparison: all letters are converted to lowercase, and punctuation runs are replaced by a single hyphen (or underscore, in the case of package filenames). For ex-

ample, `Awesome.Package`, `awesome_package`, and `awesome-package` all refer to the same project.

Project names are distinct from *import names*, the names users specify to import your code. Import names must be valid Python identifiers, so they can't have hyphens or periods and can't start with a digit. They're case-sensitive and can contain any Unicode letter or digit. As a rule of thumb, you should have a single import package per distribution package and use the same name for both (or a straightforward translation, like `random-wikipedia-article` and `random_wikipedia_article`).

Versioning Projects

The `project.version` field stores the version of your project at the time you publish the release:

```
[project]
version = "0.1"
```

The Python community has a specification for version numbers to ensure that automated tools can make meaningful decisions, such as picking the latest release of a project. At the core, versions are a dotted sequence of numbers. These numbers may be zero, and trailing zeros can be omitted: `1`, `1.0`, and `1.0.0` all refer to the same version. Additionally, you can append certain kinds of suffixes to a version ([Table 3-4](#)). The most common ones identify prereleases: `1.0.0a2` is the second alpha release, `1.0.0b3` is the third beta release, `1.0.0rc1` is the first release candidate. Each of these precedes the next, and all of them precede the final release: `1.0.0`. Python versions can use additional components as well as alternate spellings; refer to [PEP 440](#) for the full specification.

Table 3-4. Version identifiers

Release type	Description	Examples
Final release	A stable, public snapshot (default)	<code>1.0.0</code> , <code>2017.5.25</code>
Prerelease	Preview of a final release to support testing	<code>1.0.0a1</code> , <code>1.0.0b1</code> , <code>1.0.0rc1</code>
Developmental release	A regular internal snapshot, such as a nightly build	<code>1.0.0.dev1</code>
Post-release	Corrects a minor error outside of the code	<code>1.0.0.post1</code>

The Python version specification is intentionally permissive. Two widely adopted cross-language standards attach additional meaning to version numbers: [Semantic Versioning](#) uses the scheme `major.minor.patch` , where `patch` designates bugfix releases, `minor` designates compatible feature releases, and `major` designates releases with breaking changes. [Calendar Versioning](#) uses date-based versions of various forms, such as `year.month.day` , `year.month.sequence` , or `year.quarter.sequence` .

Dynamic Fields

The *pyproject.toml* standard encourages projects to define their metadata statically rather than rely on the build backend to compute the fields during the package build. Static metadata benefits the packaging ecosystem because it makes the fields accessible to other tools. It also reduces cognitive overhead: build backends use the same configuration format and populate the fields in a straightforward and transparent way.

But sometimes it’s useful to let the build backend fill in a field dynamically. For example, [“Single-Sourcing the Project Version”](#) shows how you can derive the package version from a Python module or Git tag instead of duplicating it in *pyproject.toml*.

For this reason, the project metadata standard provides an escape hatch in the form of *dynamic fields*. Projects are allowed to use a backend-spe-

cific mechanism to compute a field on the fly if they list its name under the `dynamic` key:

```
[project]  
dynamic = ["version", "readme"]
```

Many projects declare their version at the top of a Python module, like this:

```
__version__ = "0.1"
```

Updating a frequently changing item in several locations is tedious and error-prone. Some build backends, therefore, allow you to extract the version number from the code instead of repeating it in `project.version`. This mechanism is specific to your build backend, so you configure it in the `tool` table. [Example 3-5](#) demonstrates how this works with Hatch.

Example 3-5. Deriving the project version from a Python module

```
[project]
name = "random-wikipedia-article"
dynamic = ["version"] ❶

[tool.hatch.version]
path = "src/random_wikipedia_article/__init__.py" ❷

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

- ❶ Mark the version field as dynamic.
- ❷ Tell Hatch where to look for the `__version__` attribute.

You can also avoid the duplication by going in the other direction: declare the version statically in *pyproject.toml* and read it from the installed metadata at runtime, as shown in [Example 3-6](#).

Example 3-6. Reading the version from the installed metadata

```
from importlib.metadata import version

__version__ = version("random-wikipedia-article")
```


But don't go and add this boilerplate to all your projects yet. Reading the metadata from the environment isn't something you want to do during program startup. Third-party libraries like `click` perform the metadata lookup on demand—for example, when the user specifies a command-line option like `--version`. You can read the version on demand by providing a `__getattr__` function for your module ([Example 3-7](#)).⁵

Example 3-7. Reading the version from the installed metadata on demand

```
def __getattr__(name):
    if name != "__version__":
        msg = f"module {__name__} has no attribute {name}"
        raise AttributeError(msg)

    from importlib.metadata import version

    return version("random-wikipedia-article")
```

Alas, you still haven't truly single-sourced the version. Most likely, you also tag releases in your version control system (VCS) using a command like `git tag v1.0.0`. (If you don't, you should—if a release has a bug, the version tags help you find the commit that introduced it.)

Luckily, a number of build backends come with plugins that extract the version number from Git, Mercurial, and similar systems. This technique was pioneered by the `setuptools-scm` plugin. For Hatch, you can use the `hatch-vcs` plugin, which is a wrapper around `setuptools-scm` ([Example 3-8](#)).

Example 3-8. Deriving the project version from the version control system

```
[project]
name = "random-wikipedia-article"
dynamic = ["version"]

[tool.hatch.version]
source = "vcs"

[build-system]
```

```
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"
```

If you build this project from a repository and you've checked out the tag `v1.0.0`, Hatch will use the version `1.0.0` for the metadata. If you've checked out an untagged commit, Hatch will instead generate a developmental release like `0.1.dev1+g6b80314.6`. In other words, you read the project version from Git during the package build and from the package metadata at runtime.

Entry-Point Scripts

Entry-point scripts are small executables that launch the interpreter from their environment, import a module, and invoke a function (see [“Entry-point scripts”](#)). Installers like pip generate them on the fly when they install a package.

The `project.scripts` table lets you declare entry-point scripts. Specify the name of the script as the key and the module and function that the script should invoke as the value, using the format

```
package.module:function:
```

```
[project.scripts]
random-wikipedia-article = "random_wikipedia_article:main"
```

This declaration allows users to invoke the program using its given name:

```
$ random-wikipedia-article
```

The `project.gui-scripts` table uses the same format as the `project.scripts` table—use it if your application has a graphical user interface (GUI):

```
[project.gui-scripts]
random-wikipedia-article-gui = "random_wikipedia_article:gui_main"
```

Entry Points

Entry-point scripts are a special case of a more general mechanism called *entry points*. Entry points allow you to register a Python object in your package under a public name. Python environments come with a registry of entry points, and any package can query this registry to discover and import modules using the function `importlib.metadata.entry_points` from the standard library. Applications commonly use this mechanism to support third-party plugins.

The `project.entry-points` table contains these generic entry points. They use the same syntax as entry-point scripts but are grouped in subtables known as *entry-point groups*. If you want to write a plugin for another application, you register a module or object in its designated entry-point group:

```
[project.entry-points.some_application]
my-plugin = "my_plugin"
```

You can also register submodules using dotted notation, as well as objects within modules, using the format `module:object`:

```
[project.entry-points.some_application]
my-plugin = "my_plugin.submodule:plugin"
```

Let's look at an example to see how this works. Random Wikipedia articles make for fun little fortune cookies, but they can also serve as *test fixtures*⁷ for developers of Wikipedia viewers and similar apps. Let's turn the app into a plugin for the pytest testing framework. (Don't worry if you haven't worked with pytest yet; I'll cover testing in depth in [Chapter 6](#).)

Pytest allows third-party plugins to extend its functionality with test fixtures and other features. It defines an entry-point group for such plugins named `pytest11`. You can provide a plugin for pytest by registering a module in this group. Let's also add pytest to the project dependencies:

```
[project]
dependencies = ["pytest"]

[project.entry-points.pytest11]
random-wikipedia-article = "random_wikipedia_article"
```

Next, extend pytest with a test fixture returning a random Wikipedia article, as shown in [Example 3-9](#). Place this code in the `__init__.py` module of your package.

Example 3-9. Test fixture with a random Wikipedia article

```
import json
import urllib.request

import pytest

API_URL = "https://en.wikipedia.org/api/rest_v1/page/random/summary"

@pytest.fixture
def random_wikipedia_article():
    with urllib.request.urlopen(API_URL) as response:
        return json.load(response)
```

A developer of a Wikipedia viewer can now install your plugin next to pytest. Test functions use your test fixture by referencing it as a function argument (see [Example 3-10](#)). Pytest recognizes that the function argument is a test fixture and invokes the test function with the return value of the fixture.

Example 3-10. A test function that uses the random article fixture

```
# test_wikipedia_viewer.py
def test_wikipedia_viewer(random_wikipedia_article):
    print(random_wikipedia_article["extract"]) ❶
    assert False ❷
```

❶ A real test would run the viewer instead of `print()`.

- ❷ Fail the test so we get to see the full output.

You can try this out yourself in an active virtual environment in the project directory:

```
$ uv pip install .
$ py -m pytest
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: ...
plugins: random-wikipedia-article-0.1
collected 1 item

test_wikipedia_viewer.py F [100%]

===== FAILURES =====
_____ test_wikipedia_viewer _____

    def test_wikipedia_viewer(random_wikipedia_article):
        print(random_wikipedia_article["extract"])
>         assert False
E         assert False

test_wikipedia_viewer.py:4: AssertionError
----- Captured stdout call -----
Halgerda stricklandi is a species of sea slug, a dorid nudibranch, a shell-less
marine gastropod mollusk in the family Discodorididae.
===== short test summary info =====
FAILED test_wikipedia_viewer.py::test_wikipedia_viewer - assert False
===== 1 failed in 1.10s =====
```

Authors and Maintainers

The `project.authors` and `project.maintainers` fields contain the list of authors and maintainers for the project. Each item in these lists is a table with `name` and `email` keys—you can specify either of these keys or both:

```
[project]
authors = [{ name = "Your Name", email = "you@example.com" }]
```

```
maintainers = [  
    { name = "Alice", email = "alice@example.com" },  
    { name = "Bob", email = "bob@example.com" },  
]
```

The meaning of the fields is somewhat open to interpretation. If you start a new project, I recommend including yourself under `authors` and omitting the `maintainers` field. Long-lived open source projects typically list the original author under `authors`, while the people in charge of ongoing project maintenance appear as `maintainers`.

The Description and README

The `project.description` field contains a short description as a string. This field will appear as the subtitle of your project page on PyPI. Some packaging tools also use this field when displaying a compact list of packages with human-readable descriptions:

```
[project]  
description = "Display extracts from random Wikipedia articles"
```

The `project.readme` field is typically a string with the relative path to the file with the long description of your project. Common choices are *README.md* for a description written in Markdown format and *README.rst* for the reStructuredText format. The contents of this file appear on your project page on PyPI:

```
[project]  
readme = "README.md"
```

Instead of a string, you can also specify a table with `file` and `content-type` keys:

```
[project]  
readme = { file = "README", content-type = "text/plain" }
```

You can even embed the long description in the *pyproject.toml* file using the `text` key:

```
[project.readme]
content-type = "text/markdown"
text = """
# random-wikipedia-article

Display extracts from random Wikipedia articles
"""
```

Writing a README that renders well is not trivial—often, the project description appears in disparate places, like PyPI, a repository hosting service like GitHub, and inside official documentation on services like [Read the Docs](#). If you need more flexibility, you can declare the field dynamic and use a plugin like `hatch-fancy-pypi-readme` to assemble the project description from multiple fragments.

Keywords and Classifiers

The `project.keywords` field contains a list of strings that people can use to search for your project on PyPI:

```
[project]
keywords = ["wikipedia"]
```

The `project.classifiers` field contains a list of classifiers to categorize the project in a standardized way:

```
[project]
classifiers = [
    "Development Status :: 3 - Alpha",
    "Environment :: Console",
    "Topic :: Games/Entertainment :: Fortune Cookies",
]
```

PyPI maintains the [official registry of classifiers for Python projects](#). They're known as *Trove classifiers* and consist of hierarchically organized

labels separated by double colons ([Table 3-5](#)). The Trove project, initiated by Eric S. Raymond, was an early design for an open source software repository.

Table 3-5. Trove classifiers

Classifier group	Description	Example
Development Status	How mature this release is	Development Status :: 5 - Production/Stable
Environment	The environment in which the project runs	Environment :: No Input/Output (Daemon)
Operating System	The operating systems supported by the project	Operating System :: OS Independent
Framework	Any framework used by the project	Framework :: Flask
Audience	The kind of users served by the project	Intended Audience :: Developers
License	The license under which the project is distributed	License :: OSI Approved :: MIT License
Natural Language	The natural languages supported by the project	Natural Language :: English
Programming Language	The programming language the project is written in	Programming Language :: Python :: 3.12
Topic	Various topics related to the project	Topic :: Utilities

Classifiers are entirely optional. I recommend indicating the development status and the supported operating systems, which aren't covered by other metadata fields. If you'd like to include more classifiers, provide one from each classifier group.

The Project URLs

The `project.urls` table allows you to point users to your project homepage, source code, documentation, issue tracker, and similar project-related URLs. Your project page on PyPI links to these pages using the provided key as the display text for each link. It also displays an appropriate icon for many common names and URLs:

```
[project.urls]
Homepage = "https://yourname.dev/projects/random-wikipedia-article"
Source = "https://github.com/yourname/random-wikipedia-article"
Issues = "https://github.com/yourname/random-wikipedia-article/issues"
Documentation = "https://readthedocs.io/random-wikipedia-article"
```

The License

The `project.license` field is a table where you can specify your project license under the `text` key or by reference to a file under the `file` key. You may also want to add the corresponding Trove classifier for the license:

```
[project]
license = { text = "MIT" }
classifiers = ["License :: OSI Approved :: MIT License"]
```

I recommend using the `text` key with a [SPDX license identifier](#) such as “MIT” or “Apache-2.0.” The Software Package Data Exchange (SPDX) is an open standard backed by the Linux Foundation for communicating software bill of material information, including licenses.

NOTE

As of this writing, a Python Enhancement Proposal (PEP) is under discussion that changes the `license` field to a string using SPDX syntax and adds a `license-files` key for license files distributed with the package: [PEP 639](#).

If you're unsure which open source license to use for your project, choosealicense.com provides some useful guidance. For a proprietary project, it's common to specify “proprietary.” You can also add the special classifier `Private :: Do Not Upload` to prevent accidental upload to PyPI:

```
[project]
license = { text = "proprietary" }
classifiers = [
    "License :: Other/Proprietary License",
    "Private :: Do Not Upload",
]
```

The Required Python Version

Use the `project.requires-python` field to specify the versions of Python that your project supports:⁸

```
[project]
requires-python = ">=3.8"
```

Most commonly, people specify the minimum Python version as a lower bound, using a string with the format `>=3.x`. The syntax of this field is more general and follows the same rules as *version specifiers* for project dependencies (see [Chapter 4](#)).

Tools like Nox and tox make it easy to run checks across multiple Python versions, helping you ensure that the field reflects reality. As a baseline, I recommend requiring the oldest Python version that still receives security updates. You can find the end-of-life dates for all current and past Python versions on the [Python Developer Guide](#).

There are three main reasons to be more restrictive about the Python version. First, your code may depend on newer language features—for example, structural pattern matching was introduced in Python 3.10. Second, your code may depend on newer features in the standard library—look out for the “Changed in version 3.x” notes in the official documentation.

Third, it could depend on third-party packages with more restrictive Python requirements.

Some packages declare upper bounds on the Python version, such as `>=3.8, <4`. This practice is discouraged, but depending on such a package may force you to declare the same upper bound for your own package. Dependency solvers can't downgrade the Python version in an environment; they will either fail or, worse, downgrade the package to an old version with a looser Python constraint. A future Python 4 is unlikely to introduce the kind of breaking changes that people associate with the transition from Python 2 to 3.

WARNING

Don't specify an upper bound for the required Python version unless you *know* that your package is not compatible with any higher version. Upper bounds cause disruption in the ecosystem when a new version is released.

Dependencies and Optional Dependencies

The remaining fields, `project.dependencies` and `project.optional-dependencies`, list any third-party packages on which your project depends. You'll take a closer look at these fields—and dependencies in general—in the next chapter.

Summary

Packaging allows you to publish releases of your Python projects, using source distributions (*sdist*s) and built distributions (*wheels*). These artifacts contain your Python modules, together with project metadata, in an archive format that end users can easily install into their environments. The standard *pyproject.toml* file defines the build system for a Python project as well as the project metadata. Build frontends like `build`, `pip`, and `uv` use the build system information to install and run the build backend in an isolated environment. The build backend assembles an *sdist* and wheel from the source tree and embeds the project metadata. You

can upload packages to the Python Package Index (PyPI) or a private repository, using a tool like Twine. The Python project manager Rye provides a more integrated workflow on top of these tools.

- 1 Even the venerable Comprehensive Perl Archive Network (CPAN) didn't exist in February 1991, when Guido van Rossum published the first release of Python on Usenet.
- 2 By default, the `build` tool builds the wheel from the sdist instead of the source tree to ensure that the sdist is valid. Build backends can request additional build dependencies using the `get_requires_for_build_wheel` and `get_requires_for_build_sdist` build hooks.
- 3 Python's packaging ecosystem is also a great demonstration of *Conway's law*. In 1967, Melvin Conway—an American computer scientist also known for developing the concept of coroutines—observed that organizations will design systems that are copies of their communication structure.
- 4 This is especially true given the existence of *typosquatting*—where an attacker uploads a malicious package whose name is similar to a popular package—and *dependency confusion attacks*—where a malicious package on a public server uses the same name as a package on a private company repository.
- 5 This nifty technique comes courtesy of my reviewer Hynek Schlawack.
- 6 In case you're wondering, the `+g6b80314` suffix is a *local version identifier* that designates downstream changes, in this case using output from the command `git describe`.
- 7 *Test fixtures* set up objects that you need to run repeatable tests against your code.
- 8 You can also add Trove classifiers for each supported Python version. Some backends backfill classifiers for you—Poetry does this out of the box for Python versions and project licenses.